

Transformadas de scattering: invarianza sin aprendizaje

Una réplica del experimento de pocos datos de Bruna y Mallat

Andrés Felipe Barreto Farfan

Trabajo final — Matemáticas del aprendizaje de máquinas

29 de julio de 2026

Resumen

La transformada de scattering de Bruna y Mallat construye representaciones invariantes a traslaciones y estables frente a deformaciones mediante una cascada de convoluciones wavelet, módulo y promediado, sin aprender ningún parámetro. Esto sugiere una ventaja si tenemos pocos datos, donde una red convolucional tiene más capacidad que evidencia. Replicamos su experimento sobre MNIST con código propio y comparamos frente a una red convolucional entrenada con prácticas actuales, bajo un protocolo idéntico para todos los modelos y con cinco semillas por configuración. Se reprodujeron sus cifras con una diferencia media menor a 0,3 puntos, pero la ventaja del scattering resulta más estrecha de lo publicado por Bruna y Mallat, es concluyente entre 1000 y 2000 muestras y desaparece en ambos extremos, incluido $n = 300$, donde además su varianza cuadruplica la de la red.

1. Introducción

Las redes convolucionales funcionan muy bien, y no está del todo claro por qué. Una explicación atribuye su éxito a los pesos que aprenden. Otra lo atribuye a la arquitectura misma, la forma de encadenar convoluciones, no linealidades y submuestreo ya encaja con la estructura de las imágenes naturales, casi con independencia de qué filtros se usen. Saber cuál de las dos pesa más no es fácil, porque en una red corriente ambas van de la mano.

La transformada de scattering [3, 1] permite poner a prueba esa segunda hipótesis, porque es exactamente una red convolucional cuyos filtros están fijados por construcción y en la que no se aprende nada. Si una red así compete con una entrenada, entonces buena parte del mérito corresponde a la arquitectura.

Este trabajo replica el experimento central de [1] sobre MNIST y lo contrasta con una red convolucional entrenada end-to-end. Lo que nos interesa es cómo se comporta cada modelo cuando los datos escasean, que es donde la ausencia de parámetros debería notarse. Todo el código y los resultados intermedios están disponibles.¹

¹<https://github.com/afbarretof/Transformada-scattering.git>

2. La transformada de scattering

2.1. Construcción

Recogemos aquí la construcción de [1], cuyo fundamento teórico desarrolla [2], la notación es la de estos trabajos. La meta es convertir una imagen en una lista de números (un descriptor) que apenas cambie cuando la imagen se desplaza un poco o se deforma ligeramente, pero que siga distinguiendo un dígito de otro. La construcción alcanza esas dos cosas a la vez encadenando tres operaciones sencillas, convolución con wavelets, módulo y promediado.

Una *wavelet* es un filtro localizado que oscila unas pocas veces y luego decae, con media nula. Puede pensarse como un pequeño detector de un trazo de cierto grosor y cierta inclinación. Su media nula la hace ciega a un fondo constante y sensible solo a las variaciones (bordes, trazos, texturas), al convolucionarla con una imagen, la respuesta $x * \psi$ mide en cada punto cuánta oscilación de esa forma hay a su alrededor. La wavelet está concentrada en el espacio, de modo que su respuesta es local.

A partir de una única wavelet madre compleja ψ se genera un banco de filtros dilatándola y rotándola,

$$\psi_{j,\theta}(u) = 2^{-2j} \psi(2^{-j} r_{-\theta} u), \quad (1)$$

que depende de dos parámetros. La escala 2^j fija el tamaño de los detalles a los que responde el filtro, a mayor j , estructuras más grandes. La orientación θ , una de L direcciones que reparten el plano mediante la rotación r_θ , fija a lo largo de qué eje oscila. Escribimos $\lambda = (j, \theta)$ para un índice del banco de wavelets, cuyos elementos aparecen en la figura 1.

La respuesta $x * \psi_\lambda$ oscila y depende de la posición, para extraer de ella una medida estable se le aplica el módulo $|\cdot|$, que conserva la amplitud de la oscilación y descarta su fase. El operador elemental de la cascada combina ambas operaciones,

$$U[\lambda]x = |x * \psi_\lambda|, \quad (2)$$

y se itera a lo largo de un camino $p = (\lambda_1, \dots, \lambda_m)$, aplicando en cada paso una wavelet distinta seguida del módulo,

$$U[p]x = U[\lambda_m] \cdots U[\lambda_1]x. \quad (3)$$

Por último, cada camino se promedia con un filtro paso bajo ϕ_J (un filtro sin oscilación que difumina

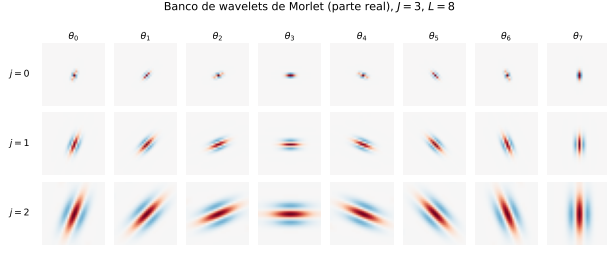


Figura 1: Banco de wavelets de Morlet (parte real) para $J = 3$ y $L = 8$. Cada fila es una escala y cada columna una orientación. Los filtros no se aprenden: se obtienen dilatando y rotando una única wavelet madre.

a la escala 2^J , borrando la posición exacta y ganando así invarianza a traslaciones), lo que produce los coeficientes de scattering:

$$S_J[p]x = U[p]x * \phi_J. \quad (4)$$

El *orden* m de un coeficiente es la longitud de su camino. El orden 0 es el simple promedio $x * \phi_J$, es decir una versión difuminada de la imagen. Un coeficiente de orden 1 resume cuánta oscilación de una escala y orientación dadas hay en cada región, cuántos bordes finos verticales, cuántos trazos gruesos diagonales, y así sucesivamente. Uno de orden 2 describe cómo se organizan esos bordes a una escala mayor (por ejemplo, si los trazos finos que detectó el primer filtro forman a su vez un patrón regular), información que el promediado del orden 1 ya había borrado.

2.2. Módulo

El promediado con ϕ_J da la invarianza a traslaciones que buscamos, pero al difuminar borra el detalle fino. Si nos quedáramos solo con él, perderíamos casi todo lo que distingue un dígito de otro. La convolución con la wavelet sí conserva ese detalle, pero tiene el defecto opuesto: se desplaza junto con la imagen, así que no es invariante. El módulo es lo que concilia ambas cosas. Como ψ es compleja, con parte real e imaginaria desfasadas un cuarto de ciclo, su módulo $|x * \psi_\lambda|$ es una envolvente, la curva suave que sigue la amplitud de la oscilación sin sus subidas y bajadas rápidas. Al ser una señal lenta, sobrevive al promediado donde la oscilación original no lo haría. Dicho de otro modo, el módulo lleva la información de detalle fino hasta un rango donde ϕ_J ya no la borra, y por eso la cascada recupera lo que un solo promediado habría perdido.

Los coeficientes de orden 1 describen la energía local por escala y orientación. Los de orden 2 capturan cómo interactúan dos escalas distintas, información que ningún promediado de orden 1 puede representar porque precisamente la promedia.

2.3. Invarianza y estabilidad

El promediado con ϕ_J hace que el descriptor apenas cambie cuando la imagen se desplaza, es casi invarian-

te a traslaciones. Pero para clasificar dígitos importa más otra propiedad, la estabilidad frente a deformaciones. Una deformación no mueve el dígito entero, lo deforma localmente (un trazo que se curva, un lazo que se ensancha). Mallat demuestra que, si la deformación es suave, el descriptor cambia poco y en proporción a lo suave que sea [2]. Eso es justo lo que hace falta para clasificar, dos versiones deformadas del mismo dígito quedan cerca en la representación. El módulo de la transformada de Fourier, en cambio, falla aquí, porque una deformación pequeña desplaza sus frecuencias altas y aleja las dos versiones.

3. Metodología

3.1. Configuración

Usamos MNIST [5] con las imágenes centradas en un lienzo de 32×32 , y una transformada de scattering con $J = 3$ (es decir $2^J = 8$, el valor que [1] selecciona por validación cruzada) y $L = 8$ orientaciones, calculada con Kymatio [4]. El descriptor resultante tiene 217 caminos sobre una rejilla espacial de 4×4 , esto es 3472 dimensiones para el orden 2 y 400 para el orden 1.

Comparamos cuatro modelos, una SVM con kernel gaussiano sobre los píxeles crudos, una SVM sobre los coeficientes de scattering, el clasificador generativo de espacios afines (PCA) de [1] y una red convolucional entrenada end-to-end.

El clasificador generativo de espacios afines (PCA) modela cada clase por separado, toma los ejemplos de un dígito, calcula su centro (el *centroide*) y las d direcciones a lo largo de las cuales esos ejemplos más varían (sus componentes principales, obtenidas por un PCA de la clase). El centro y esas direcciones definen un plano afín $A_{d,k}$, un modelo compacto de la forma típica de la clase k y de sus variaciones. Para clasificar un descriptor Sx se mide su distancia a cada plano (cuánto de Sx queda sin explicar tras proyectarlo sobre él, $\|Sx - P_{A_{d,k}}(Sx)\|$) y se le asigna la clase cuyo plano lo explica mejor. Se llama generativo porque cada clase se ajusta por su cuenta, sin trazar fronteras con las demás, al contrario que la SVM.

Los coeficientes de distintos caminos tienen magnitudes muy dispares, los de orden bajo toman valores mucho mayores que los de orden 2. Como los dos clasificadores comparan descriptores midiendo distancias, sin corregir esto un coeficiente grande pesaría más que uno pequeño por su sola magnitud, aunque el pequeño fuese más informativo. Por eso normalizamos los coeficientes antes de clasificar y, siguiendo a [1], de forma distinta según el clasificador. Para la SVM se reescala cada coeficiente al rango $[-1, 1]$. Para el clasificador afín se divide cada camino p por el mayor valor que alcanza su norma en el conjunto de entrenamiento, $\max_i \|S[p]X_i\|$, con lo que su ejemplo más intenso queda ajustado a 1.

Ahora definamos las reglas para todos los modelos.

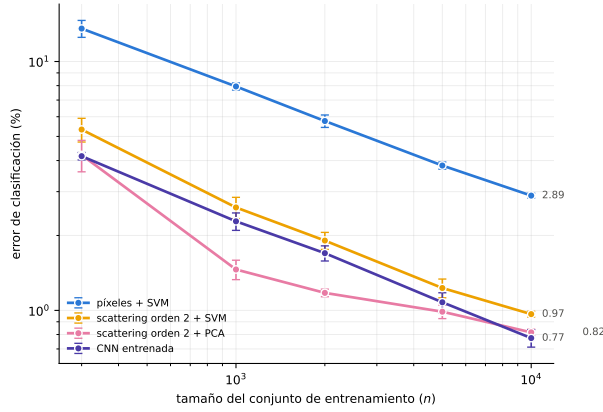


Figura 2: Error de clasificación frente al tamaño del conjunto de entrenamiento. Las barras cubren una desviación típica sobre cinco semillas. Ambos ejes en escala logarítmica.

Primero, para cada tamaño n se extrae una submuestra estratificada y los cuatro modelos reciben exactamente los mismos índices.

Segundo, validación cruzada de tres pliegues dentro de esas n muestras, seguida de un reajuste con las n completas. Esto incluye a la red, cuyo número de épocas se elige promediando las curvas de validación de los tres pliegues. Un solo conjunto de validación no servía con pocos datos, con $n = 300$ tendría apenas 60 imágenes, y entonces la época elegida dependía de si el modelo acertaba una o dos imágenes más, es decir, del azar.

Tercero, todos los números se promedian sobre cinco semillas.

4. Resultados

4.1. La réplica reproduce el original

El cuadro 1 muestra que nuestras cifras quedan próximas a las publicadas. Para el scattering de orden 2 con clasificador afín obtenemos 4,21 % frente al 4,70 % original con $n = 300$, y 0,99 % frente a 1,03 % con $n = 5000$. La coincidencia es razonable teniendo en cuenta que Kymatio no implementa la representación reducida en coseno de [1], que conserva solo la mitad de baja frecuencia de los coeficientes.

Se reproducen además dos regularidades cualitativas del original. El orden 2 mejora consistentemente al orden 1, lo que descarta que los coeficientes de segundo orden sean redundantes. Y el clasificador generativo supera a la SVM en todo el rango examinado, con la diferencia mayor donde menos datos hay. La SVM intenta aprender cómo se separan unas clases de otras, y estimar esas fronteras con pocas muestras introduce mucho error por ruido, el clasificador generativo, más rígido, no intenta estimarlas y por eso sufre menos cuando los datos escasean.

4.2. La ventaja frente a la red es estrecha

La figura 2 recoge el resultado central. Para no confundir ruido con hallazgo, solo damos por buena una diferencia entre dos modelos cuando es mayor que la suma de sus dispersiones.

Bajo ese criterio, el scattering aventaja a la red convolucional en $n = 1000$ (1,46 % frente a 2,28 %) y en $n = 2000$ (1,18 % frente a 1,70 %). En $n = 5000$ y $n = 10^4$ las barras se solapan, lo que era previsible: la ventaja debía desvanecerse al abundar los datos.

Lo que no era previsible es que también se solapen en $n = 300$, el extremo donde la ventaja debería ser máxima. Allí los promedios son indistinguibles (4,21 % frente a 4,17 %) y ocurre algo más llamativo, la dispersión del scattering es 4,3 veces la de la red ($\pm 0,61$ frente a $\pm 0,14$).

4.3. Mejoras en la red neuronal convolucional

Comparando con las cifras de 2013. Con $n = 300$, nuestra red convolucional reduce el error de la original en 3,01 puntos, mientras que nuestro scattering solo mejora al suyo en 0,49. La asimetría persiste en $n = 2000$: 0,83 puntos de mejora en la red frente a 0,12 en el scattering.

Nuestra red emplea normalización por lotes [6], una forma moderna de penalizar pesos grandes [7] y una reducción gradual del ritmo de aprendizaje a lo largo del entrenamiento. Son técnicas posteriores al artículo original y todas actúan como regularizadores, es decir, ayudan justo donde una red pequeña sufría con pocos datos. El scattering, en cambio, es el diseñado por Bruna y Mallat sin ningún tipo de mejora. Esto nos muestra que la diferencia entre los modelos se acorta porque entrenar redes convolucionales mejoró.

5. Discusión y limitaciones

Limitaciones

Actualizamos las prácticas de entrenamiento de la red, pero mantuvimos para el scattering los clasificadores del artículo original. Entonces, la comparación justa exigiría también montar un clasificador contemporáneo sobre los coeficientes de scattering, no se hizo.

Los experimentos se limitan además a MNIST, cuya variabilidad intra-clase es casi exactamente la que la teoría del scattering modela (traslaciones y deformaciones suaves), de modo que es un escenario favorable. El rango de n se detuvo en 10^4 por coste de cómputo, lo que deja sin explorar el extremo donde el artículo original sitúa su mejor cifra. Tampoco se evaluó robustez frente a deformaciones sintéticas, que es donde la estabilidad frente a deformaciones debería resultar más visible.

Tabla 1: Error de clasificación (%) en MNIST según el tamaño del conjunto de entrenamiento. Media $\pm$ desviación típica sobre cinco semillas. Las dos últimas filas reproducen los valores publicados por Bruna y Mallat para referencia.

Modelo	$n = 300$	$n = 1000$	$n = 2000$	$n = 5000$	$n = 10000$
píxeles + SVM	13.56 ± 1.06	7.94 ± 0.26	5.77 ± 0.33	3.82 ± 0.13	2.89 ± 0.06
scattering orden 1 + PCA	5.33 ± 0.20	2.85 ± 0.28	2.21 ± 0.08	1.91 ± 0.11	1.82 ± 0.05
scattering orden 1 + SVM	7.26 ± 0.59	3.56 ± 0.37	2.49 ± 0.19	1.69 ± 0.16	1.26 ± 0.07
scattering orden 2 + PCA	4.21 ± 0.61	1.46 ± 0.13	1.18 ± 0.04	0.99 ± 0.06	0.82 ± 0.02
scattering orden 2 + SVM	5.33 ± 0.58	2.59 ± 0.25	1.91 ± 0.15	1.23 ± 0.11	0.97 ± 0.03
CNN entrenada	4.17 ± 0.14	2.28 ± 0.18	1.70 ± 0.12	1.08 ± 0.10	0.77 ± 0.06
<i>Bruna y Mallat, scattering orden 2 + PCA</i>	4.70	2.30	1.30	1.03	0.88
<i>Bruna y Mallat, ConvNet</i>	7.18	3.21	2.53	1.52	0.85

Conteo de parámetros

La representación de scattering no aprende parámetros, pero el clasificador que se monta encima sí estima números a partir de los datos, entre $2,1 \cdot 10^5$ y $4,9 \cdot 10^6$ según la dimensión d seleccionada, frente a los $7,6 \cdot 10^4$ pesos entrenables de nuestra red convolucional. Contando todo, el pipeline de scattering maneja *más* números.

La diferencia está en cómo se obtienen. Los números del clasificador afín se calculan clase por clase, cada una por separado y con una fórmula cerrada, sin que la solución de una clase dependa de las demás. Los pesos de la red, en cambio, se ajustan todos a la vez, y cada uno depende de todos los demás. Esa dependencia mutua es lo caro de estimar.

6. Conclusiones

Se replicó sobre MNIST el experimento de pocos datos de Bruna y Mallat con código propio, y las cifras coinciden con las suyas dentro de unas pocas decimas. La hipótesis central, que una representación sin parámetros aprendidos mejora a una red entrenada cuando hay pocos datos, se sostiene, pero en un margen más estrecho de lo que sugiere el artículo original, es clara entre 1000 y 2000 muestras y se desvanece en los dos extremos.

Por un lado, la ventaja del scattering se ha reducido frente a 2013, y casi todo el cambio viene de que entrenar redes convolucionales mejoró. Por otro lado, con muy pocos datos el error es tan variable (cuatro veces el de la red) que deja de ser confiable.

Referencias

- [1] J. Bruna y S. Mallat, “Invariant scattering convolution networks”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, n.º 8, pp. 1872–1886, 2013.
- [2] S. Mallat, “Group invariant scattering”, *Communications on Pure and Applied Mathematics*, vol. 65, n.º 10, pp. 1331–1398, 2012.

- [3] J. Bruna y S. Mallat, “Classification with scattering operators”, en *IEEE Conference on Computer Vision and Pattern Recognition*, 2011.
- [4] M. Andreux *et al.*, “Kymatio: Scattering transforms in Python”, *Journal of Machine Learning Research*, vol. 21, n.º 60, pp. 1–6, 2020.
- [5] Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, “Gradient-based learning applied to document recognition”, *Proceedings of the IEEE*, vol. 86, n.º 11, pp. 2278–2324, 1998.
- [6] S. Ioffe y C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift”, en *International Conference on Machine Learning*, 2015.
- [7] I. Loshchilov y F. Hutter, “Decoupled weight decay regularization”, en *International Conference on Learning Representations*, 2019.